



COE304 Embedded Systems Project Proposal
Spring 2025
Hardware and Software Integration (Phase 3)

Project Name: One Robot, Endless Possibilities | [Speak](#), [Control](#), [Avoid](#)

Group No: 10

	Student ID	Student Name
1.	210901038	Beyza Nur Yazıcı
2.	210901016	İrem Batıgün
3.	220901646	Misem Mohamed

1. Software that meets your Requirements

Write the code for your project in C or Assembly to achieve the second stage requirements.
Be specific for each one. Fill the following table.

RQ1	1. Measure distance using ultrasonic sensor.
	Example: This requirement has been achieved by the following code which triggers the ultrasonic sensor and reads the echo pulse duration to calculate distance in centimeters. <pre>// Trigger ultrasonic pulse and read echo time, return distance in cm int readUltrasonic() { digitalWrite(TRIG_PIN, LOW); delayMicroseconds(4); digitalWrite(TRIG_PIN, HIGH); delayMicroseconds(10); digitalWrite(TRIG_PIN, LOW); long duration = pulseIn(ECHO_PIN, HIGH); // Measure echo pulse width return duration / 29 / 2; // Convert microseconds to cm }</pre>

RQ2	2. Control servo motor to scan left, right, and center positions smoothly. Example: This requirement has been achieved by smoothly moving the servo to a target angle one degree at a time with a small delay, ensuring smooth motion. <pre>// Smoothly increment or decrement servo until reaching targetAngle void smoothServoWrite(int targetAngle) { int current = servo.read(); // Get current servo angle while (current != targetAngle) { if (current < targetAngle) current++; else current--; servo.write(current); // Move one degree at a time delay(5); // Small delay for smooth motion } }</pre>
RQ3	3. Perform autonomous obstacle avoidance by detecting obstacles and turning accordingly. Example: This requirement is met by stopping if an obstacle is closer than 12 cm, backing up, scanning left and right distances, then turning toward the side with more space. <pre>// Continuously check distance ahead and steer around obstacles void autonomousObstacleAvoidance() { int frontDistance = readUltrasonic(); // Measure distance ahead if (frontDistance <= 12) { // If obstacle is closer than 12 cm stopMotion(); // Stop forward motion moveBackward(); // Back up slightly delay(100); stopMotion(); // Scan left and right distances by moving servo int leftDistance = scanAt(LEFT_ANGLE); int rightDistance = scanAt(RIGHT_ANGLE); smoothServoWrite(CENTER_ANGLE); // Re-center servo delay(200); // Turn toward the side with more free space if (leftDistance > rightDistance) { Serial.println("Turning left"); turnLeft(); } else { Serial.println("Turning right"); turnRight(); } } }</pre>

	<pre> delay(TURN_DURATION_MS); // Run turn long enough for ~90° stopMotion(); delay(200); } else { moveForward(); // No obstacle → keep moving forward } } </pre>
RQ4	4. Control four DC motors for forward, backward, left, right, and stop motions. Example: This requirement is fulfilled by functions that run the motors in the appropriate directions using the Adafruit Motor Shield library. <pre> // Drive all four motors forward void moveForward() { motor1.run(FORWARD); motor2.run(FORWARD); motor3.run(FORWARD); motor4.run(FORWARD); } // Drive all four motors backward void moveBackward() { motor1.run(BACKWARD); motor2.run(BACKWARD); motor3.run(BACKWARD); motor4.run(BACKWARD); } // Spin in place to the right: left wheels backward, right wheels forward void turnRight() { motor1.run(BACKWARD); motor2.run(BACKWARD); motor3.run(FORWARD); motor4.run(FORWARD); } // Spin in place to the left: left wheels forward, right wheels backward void turnLeft() { motor1.run(FORWARD); motor2.run(FORWARD); motor3.run(BACKWARD); motor4.run(BACKWARD); } // Stop all motors immediately void stopMotion() { motor1.run(RELEASE); motor2.run(RELEASE); motor3.run(RELEASE); motor4.run(RELEASE); } </pre>

RQ5	5. Accept manual movement commands via serial input and execute corresponding actions. Example: This requirement is met by reading serial input characters and calling movement functions based on the command received. <pre> void loop() { // If a byte arrives on Serial, handle it as a manual command if (Serial.available() > 0) { char cmd = Serial.read(); // Read single character Serial.print("Command received: "); Serial.println(cmd); handleCommand(cmd); // Execute manual control } else { autonomousObstacleAvoidance(); // No command → do obstacle avoidance } } // Interpret Bluetooth or voice command characters void handleCommand(char c) { switch (c) { case 'F': case '^': // 'F' or '^' means go forward Serial.println("Forward"); moveForward(); break; case 'B': case '-': // 'B' or '-' means go backward Serial.println("Backward"); moveBackward(); break; case 'L': case '<': // 'L' or '<' means turn left Serial.println("Left"); turnLeft(); break; case 'R': case '>': // 'R' or '>' means turn right Serial.println("Right"); turnRight(); break; case 'S': case '*': // 'S' or '*' means stop Serial.println("Stop"); </pre>

	<pre>stopMotion(); break; default: // Any other char → emergency stop stopMotion(); break; } }</pre>
RQ6	6. Set motor speed to a fixed PWM value for consistent movement.
	Example: This requirement is achieved by setting the speed of all motors to 180 during setup. <pre>#define MOTOR_SPEED 180 // PWM speed for all four DC motors (0–255) // Initialize all motors at defined speed motor1.setSpeed(MOTOR_SPEED); motor2.setSpeed(MOTOR_SPEED); motor3.setSpeed(MOTOR_SPEED); motor4.setSpeed(MOTOR_SPEED);</pre>
RQ7	7. Provide serial feedback on received commands.
	Example: This requirement is met by printing the received command and action to the serial monitor. <pre>char cmd = Serial.read(); // Read single character Serial.print("Command received: "); Serial.println(cmd);</pre>
RQ8	8. Provide a default safe state when unknown commands are received.
	Example: The system stops all motors immediately if an unrecognized command is received, preventing unsafe behavior. <pre>default: // Any other char → emergency stop stopMotion(); break;</pre>
RQ9	9. Use delays appropriately to allow hardware to settle and avoid timing issues.
	Example: Delays after servo movement and motor commands ensure sensors and actuators have time to respond.

	<pre>// Wait after servo moves before reading sensor to Run turn long enough for ~90° delay(TURN_DURATION_MS); // Allow servo to settle delay(SCAN_DELAY_MS);</pre>
RQ10	10. Modularize code for clarity and maintainability. Example: Each functionality (movement, scanning, command handling) is encapsulated in dedicated functions, improving readability and testability. <pre>void scanAt(int angle) { ... } void smoothServoWrite(int targetAngle) {...} void handleCommand(char c) { ... } void autonomousObstacleAvoidance() {...} void moveForward() { ... } void moveBackward() {...} void turnRight() {...} void turnLeft() {...} void stopMotion() {...}</pre>
RQ11	11. System shall operate continuously without manual reset. Example: The loop() function runs indefinitely, continuously checking for commands or performing autonomous navigation. <pre>void loop() { // If a byte arrives on Serial, handle it as a manual command if (Serial.available() > 0) { char cmd = Serial.read(); // Read single character Serial.print("Command received: "); Serial.println(cmd); handleCommand(cmd); // Execute manual control } else { autonomousObstacleAvoidance(); // No command → do obstacle avoidance } }</pre>

2. HW and SW Integration

Integrate the hardware components with the Arduino board and uploading the code to the board for testing. Write down any challenge or error you faced and how did you solve it.

	Challenge or Error	How did you solve it?
1.	Bluetooth Module Availability and Hardware Compatibility	Initially, we were unable to find the Bluetooth module in local markets, as it was frequently out of stock. As a temporary solution, we borrowed an ESP32 Wi-Fi + Bluetooth module. Upon researching its specifications, we found that it offered superior performance compared to the HC-05. However, it also required a more complex wiring setup, which did not align well with our existing design. To maintain our original hardware layout, we eventually sourced an HC-05 module. This module also required soldering in order to interface with the motor driver shield, but compared to the DC motor connections, the soldering process was significantly simpler and more straightforward.
2.	Encoder Disc Absence	To address the encoder disk shortage, we carefully measured the only available encoder disk and recreated its exact geometry using ShapeWorks. After finalizing the design, we utilized a 3D printer to fabricate four new encoder disks based on the model. This approach allowed us to successfully replicate all the encoder disks, using just a single physical sample as a reference. However, we initially implemented the encoder without thoroughly researching its purpose and functionality. Later, we realized that our project does not require advanced features such as: • Precise distance measurement (e.g., moving exactly 30 cm), • Linear positioning or navigation, • Speed control or PID motor control, • Simultaneous Localization and Mapping (SLAM), • Odometric position tracking. Despite this, we decided to retain the newly designed encoder disks in our build to support future development and potential feature expansion. By doing so, we ensured our design remains flexible and scalable should more complex capabilities be added later.

3.	Lack of Screws	The package we received was missing the screws required to mount the motors, which caused difficulties in securing them to the chassis. There were several alternative solutions, such as using super glue, tying the motors to the chassis with plastic cable ties, or sourcing replacement screws. Ultimately, we chose to purchase or borrow the correct screws, as this was the most reliable and durable option for ensuring mechanical stability.
4.	Shipping Damage on L293D Shield	The pins of the L293D motor driver shield arrived bent due to shipping damage. To straighten them without causing further damage, we used a pair of needle-nose pliers. This allowed us to carefully realign the bent pins without breaking or weakening them.
5.	A Detached Ground Wire In the Servo Motor	Although we tested it and confirmed it was working, we decided to use a brand-new servo, as we believed the condition of the wire could lead to potential inconsistencies.
6.	Stability of the Ultrasonic Sensor and Servo Motor	The ultrasonic sensor was required to be mounted on top of the servo motor, making the servo act as the structural base. Therefore, it needed to be firmly secured. We achieved this by fastening it to the car chassis using multiple screws and the pre-drilled mounting holes. To mount the ultrasonic sensor itself, we utilized a dedicated holder, which allowed attachment to the servo using double-sided adhesive tape. However, ensuring mechanical balance was challenging, the setup frequently tipped forward or backward during operation, requiring repeated adjustments to maintain stability.
7.	Not pre-wired DC Motors	The DC motors we received did not come with pre-attached wires, so we had to manually solder the connections. In order to perform proper soldering, we first familiarized ourselves with basic soldering techniques, including tinning the wire tips, preheating the joint area, and applying the solder to the heated surfaces rather than the iron directly. However, during the initial soldering process, we overlooked critical aspects such as wire polarity and color orientation. This oversight led to a wiring mismatch, which we only identified later when we faced difficulties connecting the motors to the motor driver . As a result, we had to desolder the initial connections and perform a second round of soldering , this time ensuring correct polarity alignment and mechanically stable joints to guarantee reliable motor driver operation

8.	Challenges in Connecting DC Motors to Motor Shield	Connecting the DC motors to the motor shield proved to be a time-consuming task, mainly due to the lack of clear and consistent tutorials or documentation. We struggled to determine the correct wiring order and polarity for the motor terminals, and it was unclear how critical the wire sequence was. In some reference materials, the middle terminal was shown as a pinned GND, making it inaccessible—yet in our version, the layout differed, further adding to the confusion. We were forced to try multiple wiring combinations through trial and error to identify the correct configuration. Additionally, due to inconsistent soldering and poor tip shaping on the wire ends, inserting the wires into the shield terminals was physically difficult, often resulting in unstable or unreliable connections. During this process, the wheels often rotated in the wrong direction or failed to move altogether, which significantly delayed our progress.
9.	Upload Failure Due to Serial Pin Interference (D0/D1 Conflict)	After completing our connections and attempting to upload the code to the Arduino, the upload consistently failed despite all our validations. Upon further investigation, we discovered the issue was caused by a common mistake: something was connected to pins D0 (RX) and D1 (TX) during the upload process. These pins are used for serial communication between the Arduino and the computer and must remain unconnected when uploading code. In our case, the Bluetooth module was still attached to those pins, which interfered with the upload process.
10.	Servo Malfunction Due to Timer Conflict on Motor Shield	We initially connected our servo motor to the SERVO_1 port on the motor shield, but it didn't function correctly. Upon further research, we found that on motor shields like the Adafruit Motor Shield v1/v2, certain pins serve special purposes. The SERVO_1 and SERVO_2 ports are designated for servo output and are compatible with the Servo library. However, pins like D9 and D10 can lead to conflicts when used with both motors and servos, as they share the same internal timer . In our case, connecting the servo to SERVO_1 (Arduino pin 10) caused such a conflict. The recommended solution was to switch the connection to SERVO_2 (Arduino pin 9) , which is better suited for servo control without timing conflicts. After doing so, the servo operated as expected.
11.	Connection problem with Bluetooth module	The Bluetooth module we purchased did not ask for a PIN code on first use and could not be configured in any way. After researching, we found that it might be a clone, which can cause

		such issues. Therefore, we borrowed another module, but even though this one was original, we still faced connection problems. When we couldn't solve the issue ourselves, we asked you for help, and you showed us that Bluetooth can often cause unexpected issues — there are even special debugging tools made just for this. You reset the Bluetooth pairing on the phone and re-paired it, which resolved the problem. Since then, we have not experienced any further issues.
12.	Forget to buy a switch – the robot couldn't be stopped without removing the battery, and autonomous behavior was disrupted	We solved this purely in software with two approaches: 1. On-demand autonomy – In loop() we check Serial.available(). If there's input, we call handleCommand(cmd) (forward/back/left/right/stop); if not, we call autonomousObstacleAvoidance(). This both disables obstacle avoidance during manual control and provides an emergency-stop path without pulling the battery. 2. Explicit modes – Maintain a mode variable (Obstacle / Bluetooth / Voice) and switch on special commands. We rejected this due to added state-machine complexity. We chose option 1 for its simplicity, seamless hand-off, built-in emergency stop, and easier debugging (all logic in one control flow).

3. Testing and Debugging:

Emphasize the importance of testing each component and functionality of the project code thoroughly to identify and fix any errors or bugs (if any). You can add photos or videos.

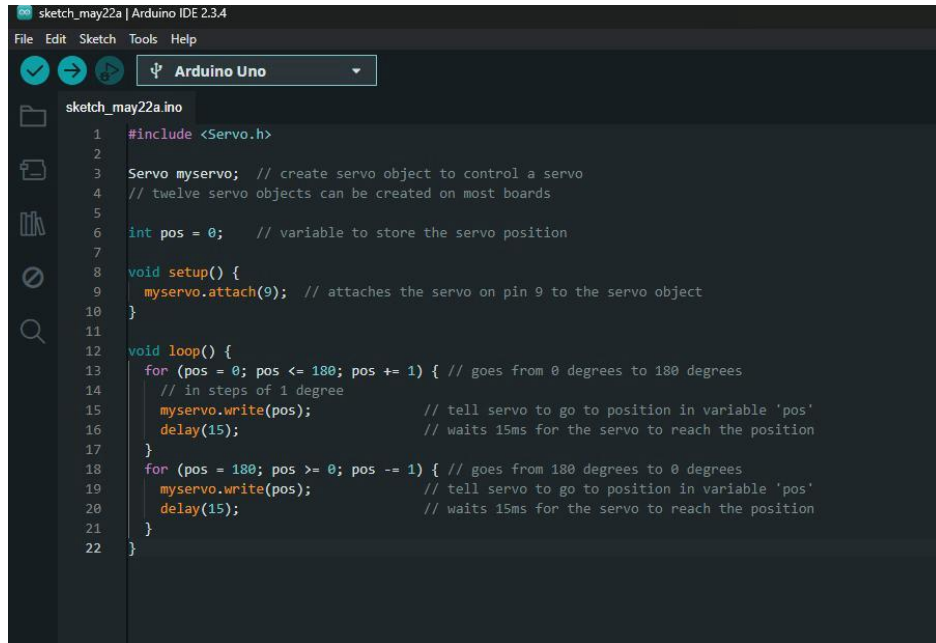
Thorough testing and debugging of each component is a critical step in embedded systems development. It ensures system reliability, correct behavior, function and smooth integration of all modules. In our project, we followed a modular approach to verify the functionality of each part individually before combining them.

1. Servo Motor Testing

We began by testing the servo motor, which is responsible for carrying the ultrasonic sensor to scan the environment left and right for obstacle detection. We wrote a script to sweep the servo from 0° to 180° and back in 1° increments. This allowed us to confirm that the servo responds smoothly and consistently to angle commands.

Observation: The motor arm rotated as expected, demonstrating that the pin connection (D9), PWM signal, and power supply were functioning correctly.

Result: No jittering or delays were observed, verifying that the servo could later be used for scanning in the robotic car.



```
1 #include <Servo.h>
2
3 Servo myservo; // create servo object to control a servo
4 // twelve servo objects can be created on most boards
5
6 int pos = 0;    // variable to store the servo position
7
8 void setup() {
9   myservo.attach(9); // attaches the servo on pin 9 to the servo object
10 }
11
12 void loop() {
13   for (pos = 0; pos <= 180; pos += 1) { // goes from 0 degrees to 180 degrees
14     // in steps of 1 degree
15     myservo.write(pos);              // tell servo to go to position in variable 'pos'
16     delay(15);                       // waits 15ms for the servo to reach the position
17   }
18   for (pos = 180; pos >= 0; pos -= 1) { // goes from 180 degrees to 0 degrees
19     myservo.write(pos);              // tell servo to go to position in variable 'pos'
20     delay(15);                       // waits 15ms for the servo to reach the position
21   }
22 }
```

Link to video with test result of **Servo Motor**: https://istinye-my.sharepoint.com/:v/g/personal/210901038_stu_istinye_edu_tr/EW9BHaciSRtEtWb2v8HnXpIBxT5TJh-lczQ1BDwhIgzclA?e=UNVUGB

2. Ultrasonic Sensor Testing

Next, the ultrasonic sensor (HC-SR04) was tested independently to verify its ability to accurately detect distances. A simple Arduino sketch was used to trigger the sensor and display real-time distance measurements on the Serial Monitor. This test helped identify any potential wiring or signal issues before integrating the sensor into the full system.

Observation: When a stick was moved closer, the values on the monitor changed in real time.

Result: The sensor accurately detected distances, ranging from a few centimeters to over 400 cm.

<pre> sketch_may22a.ino 1 const int trigPin = 3; 2 const int echoPin = 2; 3 float duration, distance; 4 void setup() { 5 pinMode(trigPin, OUTPUT); 6 pinMode(echoPin, INPUT); 7 Serial.begin(9600); 8 } 9 void loop() { 10 digitalWrite(trigPin, LOW); 11 delayMicroseconds(2); 12 digitalWrite(trigPin, HIGH); 13 delayMicroseconds(10); 14 digitalWrite(trigPin, LOW); 15 duration = pulseIn(echoPin, HIGH); 16 distance = (duration*0.0343)/2; 17 Serial.print("Distance: "); 18 Serial.println(distance); 19 delay(100); 20 } 21 </pre>	<pre> Distance: 416.11 Distance: 416.09 Distance: 415.77 Distance: 417.41 Distance: 415.75 Distance: 415.75 Distance: 3.07 Distance: 6.69 Distance: 4.87 Distance: 3864.75 Distance: 3.07 Distance: 3871.65 Distance: 5.78 Distance: 432.78 Distance: 416.88 Distance: 415.63 Distance: 416.09 Distance: 415.22 Distance: 418.61 Distance: 415.24 Distance: 416.11 </pre>
--	---

Link to video with test result of Ultrasonic Sensor : https://istinye-my.sharepoint.com/:v/g/personal/210901038_stu_istinye_edu_tr/EaSmJB6na35IiEGWLtGp92YBGHqjrj38f1gEFiwWb9j1QCA?e=X7zbWd

3. Bluetooth Module Testing

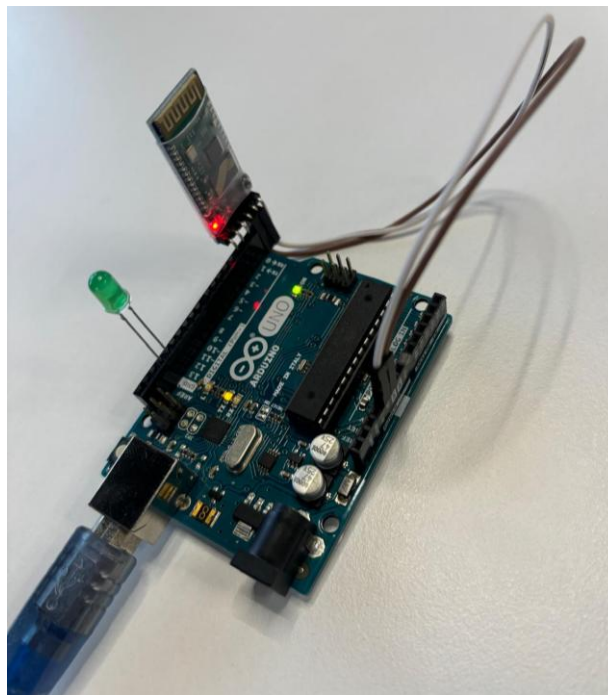
Bluetooth module (HC-05), which facilitates wireless control of the robotic car via smartphone. This test was essential to ensure stable serial communication and proper pairing functionality before system integration.

Test Method: We used a simple program to turn an LED on/off by sending '1' and '0' via the serial monitor or Bluetooth terminal app.

Observation: The green LED toggled successfully in response to commands. The RX/TX communication with the HC-05 was verified through the blinking onboard LED on the module.

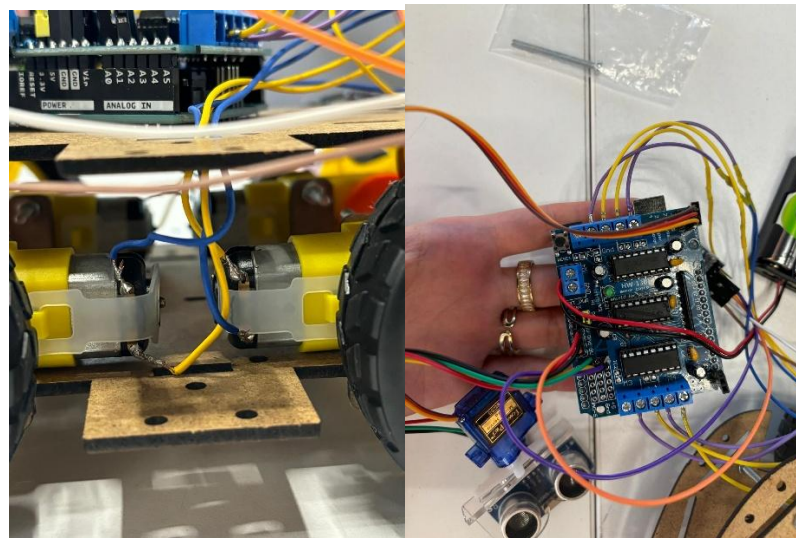
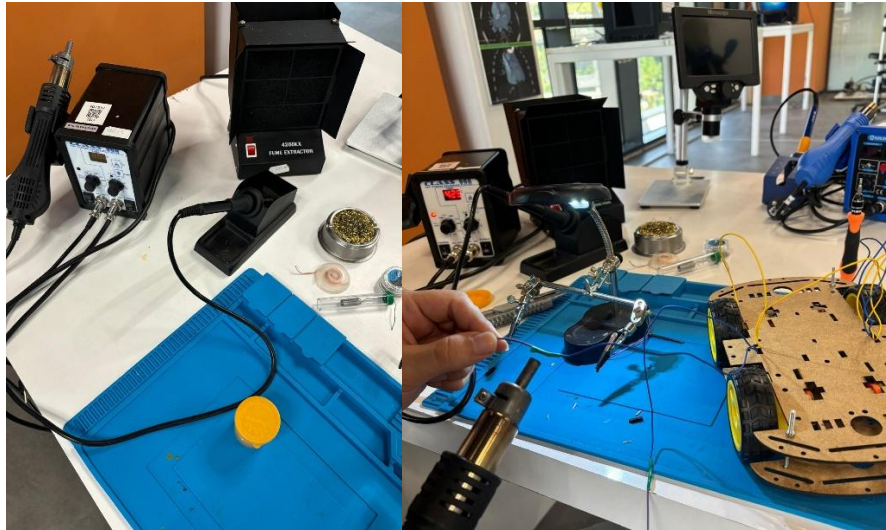
Result: This confirmed successful Bluetooth pairing and serial data reception.

```
sketch_may22a.ino
1  #define ledPin 13
2  int state = 0;
3
4  void setup() {
5      pinMode(ledPin, OUTPUT);
6      digitalWrite(ledPin, LOW);
7      Serial.begin(9600);
8  }
9
10 void loop() {
11     if(Serial.available() > 0){ // Checks whether data is coming from the serial port
12         state = Serial.read(); // Reads the data from the serial port
13     }
14
15     if (state == '0') {
16         digitalWrite(ledPin, LOW); // Turn LED OFF
17         state = 0;
18     }
19     else if (state == '1') {
20         digitalWrite(ledPin, HIGH);
21         state = 0;
22     }
23 }
```



4. DC Gear Motor Testing

The DC motors arrived without wires, so we soldered wires to the motor terminals to enable connectivity. Then, we soldered the other ends of the wires to allow connection to the motor shield. This ensured a reliable and durable connection between the motors and the shield for consistent power delivery and control.



To validate the movement and control logic of the robotic car, we performed isolated testing of the four DC gear motors.

Test Procedure:

We created a loop that commands the car to:

Turn **right** for 5 seconds, turn **left** for 5 seconds, move **backward** for 5 seconds and move **forward** for 5 seconds. Between each action, the car stops momentarily to ensure clear transitions.

Observations: All motors responded correctly to their respective direction commands. The car performed each movement (turns and straight motion) accurately without any delay or missed signals.

Serial prints (e.g., "Turning left", "Moving forward") confirmed the execution flow in real-time through the Serial Monitor.

Results: Motor polarity and wiring are correctly configured.

The robotic car is physically capable of executing the full set of movement commands (forward, backward, left, right, stop).

```
test_code_for_motors$
1 #include <AFMotor.h>
2
3 #define Speed 255
4
5 AF_DCMotor M1(1);
6 AF_DCMotor M2(2);
7 AF_DCMotor M3(3);
8 AF_DCMotor M4(4);
9
10 void setup() {
11   Serial.begin(9600);
12   M1.setSpeed(Speed);
13   M2.setSpeed(Speed);
14   M3.setSpeed(Speed);
15   M4.setSpeed(Speed);
16 }
17
18 void loop() {
19   // Turn right(5 seconds)
20   Serial.println("Turning right");
21   right();
22   delay(5000);
23   Stop();
24   delay(1000); // wait for 1 seconds
25
26   // Turn left (5 seconds)
27   Serial.println("Turning left");
28   left();
29   delay(5000);
30   Stop();
31   delay(1000);
32
33   // Move backward(5 seconds)
34   Serial.println("Moving backward");
35   backward();
36   delay(5000);
37   Stop();
38   delay(1000);
39
40
41
42 // Move forward(5 seconds)
43 Serial.println("Moving forward");
44 forward();
45 delay(5000);
46 Stop();
47
48 delay(3000); // wait for 3 seconds in each loop
49 }
50
51 // Fuctions:
52 void forward() {
53   M1.run(FORWARD);
54   M2.run(FORWARD);
55   M3.run(FORWARD);
56   M4.run(FORWARD);
57 }
58
59 void backward() {
60   M1.run(BACKWARD);
61   M2.run(BACKWARD);
62   M3.run(BACKWARD);
63   M4.run(BACKWARD);
64 }
65
66 void right() {
67   M1.run(BACKWARD);
68   M2.run(BACKWARD);
69   M3.run(FORWARD);
70   M4.run(FORWARD);
71 }
72
73 void left() {
74   M1.run(FORWARD);
75   M2.run(FORWARD);
76   M3.run(BACKWARD);
77   M4.run(BACKWARD);
78 }
79
80 void Stop() {
81   M1.run(RELEASE);
82   M2.run(RELEASE);
83   M3.run(RELEASE);
84   M4.run(RELEASE);
85 }
```

Link to video with test result of DC Gear Motor : https://istinye-my.sharepoint.com/:v:/g/personal/210901038_stu_istinye_edu_tr/EY8MHk5t5SxEvHujQW8FcQEBdrNx3HKMUmJ0mkOIEWiACQ?e=WEb3mQ

5. Battery and Power Supply Testing

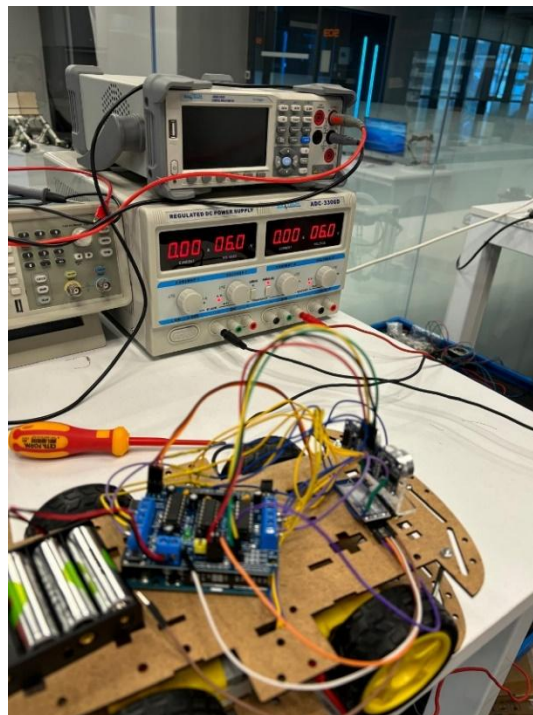
Power supply is one of the **most critical elements** in embedded and mobile robotics projects, as it directly affects the **reliability, performance, and safety** of the entire system. For our robotic car, which integrates all components of the robotic car, **stable and sufficient power** is essential for proper, consistent and safe operation. Before integrating the battery pack into the mobile system, we conducted controlled testing using a **regulated DC power supply** to simulate expected voltage loads.

In the lab setup (as shown in the photo), we powered the system using a **bench power supply set to 6V**. This allowed us to monitor the real-time voltage and current drawn by the motors and logic components. The multimeter ensured that voltage fluctuations reducing the risk of hardware damage.

After confirming that the system could operate stably at 6V, we switched to a **battery pack (4x1.5V AA batteries)** as the portable power source for the car. The battery output was verified to match the voltage requirements of the motor driver and the Arduino board. Stable operation from both power sources confirmed that our wiring, voltage levels, and consumption were within safe limits.

Key Checks Performed:

- Verified 6V from the lab power supply to match motor and controller requirements.
- Ensured all motors responded with sufficient torque under load.



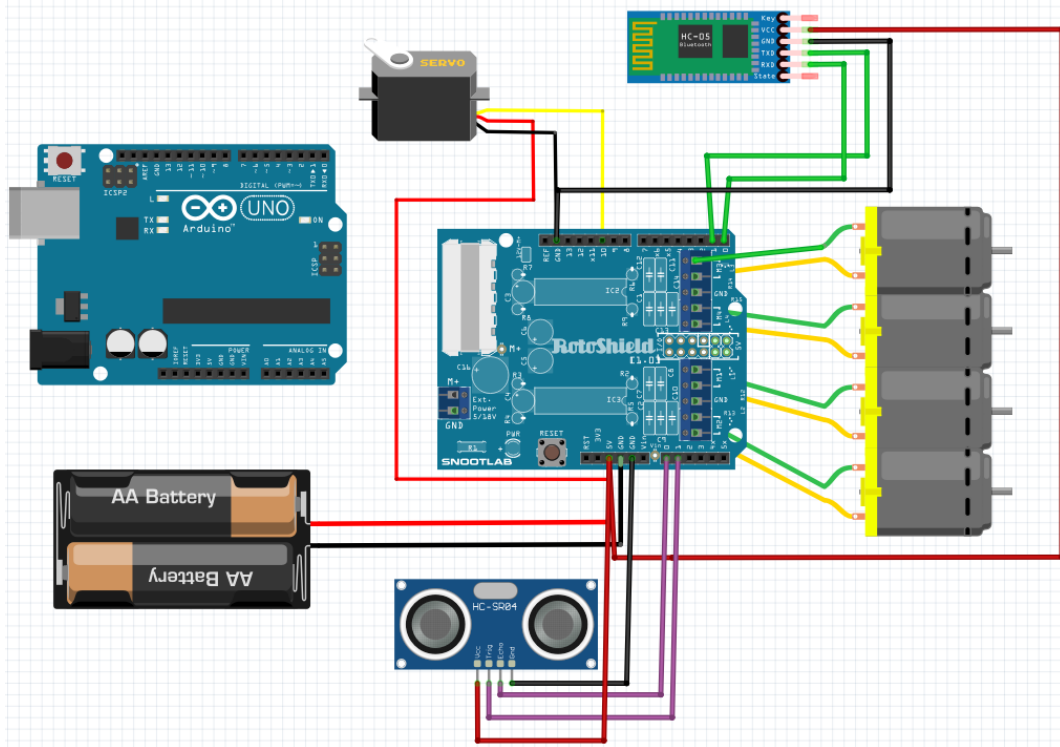
4. Documentation: document your project, including

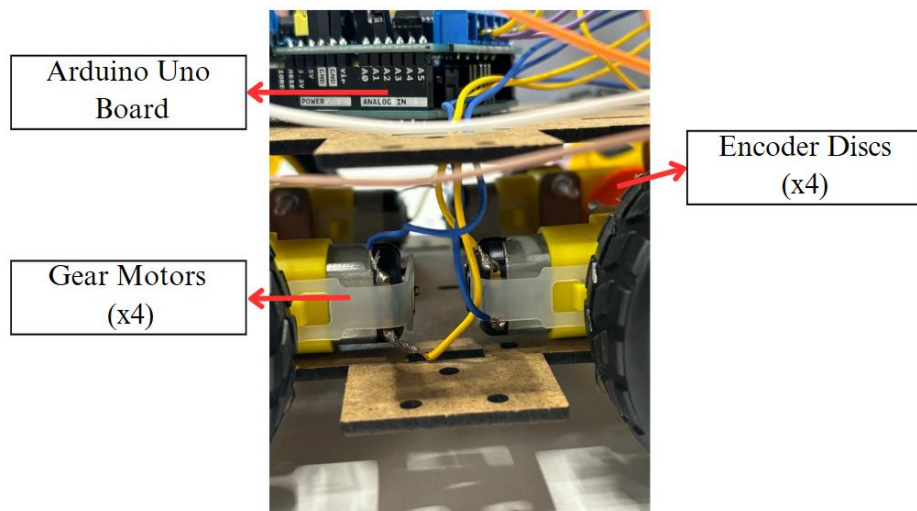
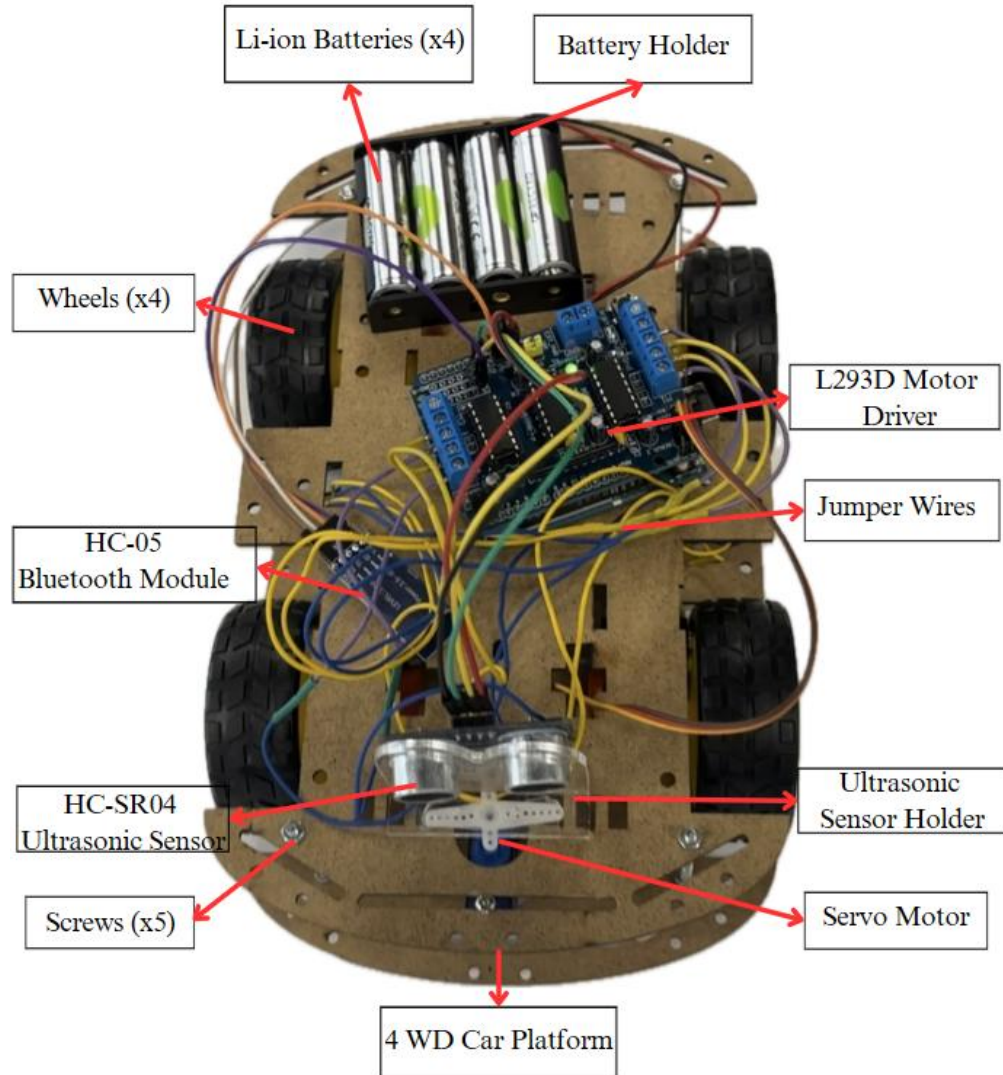
- **The circuit diagram**

We constructed our robotic car system according to the Fritzing circuit diagram we made in the 2nd Satge of the Project. This schematic guided the correct wiring and integration of all major components, including the Arduino Uno board, HC-05 Bluetooth module, servo motor, ultrasonic sensor, L293D motor driver shield, and gear motors powered by four Li-ion batteries.

Each connection was carefully made to ensure proper communication and power delivery between the components. The motor driver shield was placed on top of the Arduino Uno to control the four DC gear motors, while the Bluetooth module enabled wireless control. The ultrasonic sensor, mounted on a servo motor, provided real-time environmental scanning for obstacle avoidance.

In addition to the circuit schematic, we have also included real images of the final assembled car. These images highlight the physical placement of all components, including the battery holder, sensor modules, wiring setup, and chassis structure. This final prototype reflects a successful translation of our digital design into a functional embedded system.





- **Code explanation with comments**

```

• #include <Servo.h>           // Include Servo library for controlling the servo motor
• #include <AFMotor.h>        // Include Adafruit Motor Shield library for DC motors
•
• // Pin definitions
• #define ECHO_PIN            A0    // Ultrasonic sensor echo pin
• #define TRIG_PIN            A1    // Ultrasonic sensor trigger pin
• #define SERVO_PIN           9     // Servo signal pin (Arduino D9)
• #define MOTOR_SPEED         180   // PWM speed for all four DC motors (0-255)
•
• // Servo angles and timing constants
• #define LEFT_ANGLE           150   // Servo angle to scan left side
• #define RIGHT_ANGLE          30    // Servo angle to scan right side
• #define CENTER_ANGLE         103   // Servo "home" or center position
• #define SCAN_DELAY_MS        400   // Delay after moving servo before reading distance
• #define TURN_DURATION_MS     450   // How long to run motors to achieve ~90° turn
•
• // Create global objects
• Servo servo;                  // Servo object
• AF_DCMotor motor1(1),         // Motor shield channel 1
•             motor2(2),         // Motor shield channel 2
•             motor3(3),         // Motor shield channel 3
•             motor4(4);         // Motor shield channel 4
•
• void setup() {
•     Serial.begin(9600);        // Start serial communication at 9600 baud
•     pinMode(TRIG_PIN, OUTPUT); // Set trigger pin as output
•     pinMode(ECHO_PIN, INPUT);  // Set echo pin as input
•
•     servo.attach(SERVO_PIN);    // Attach servo to its pin
•     smoothServoWrite(CENTER_ANGLE); // Move servo to center on startup
•
•     // Initialize all motors at defined speed
•     motor1.setSpeed(MOTOR_SPEED);
•     motor2.setSpeed(MOTOR_SPEED);
•     motor3.setSpeed(MOTOR_SPEED);
•     motor4.setSpeed(MOTOR_SPEED);
• }
•
• void loop() {
•     // If a byte arrives on Serial, handle it as a manual command
•     if (Serial.available() > 0) {
•         char cmd = Serial.read(); // Read single character

```

```
• Serial.print("Command received: ");
• Serial.println(cmd);
• handleCommand(cmd); // Execute manual control
• }
• else {
•     autonomousObstacleAvoidance(); // No command → do obstacle avoidance
• }
• }
•
• // Interpret Bluetooth or voice command characters
• void handleCommand(char c) {
•     switch (c) {
•         case 'F': case '^': // 'F' or '^' means go forward
•             Serial.println("Forward");
•             moveForward();
•             break;
•
•         case 'B': case '-': // 'B' or '-' means go backward
•             Serial.println("Backward");
•             moveBackward();
•             break;
•
•         case 'L': case '<': // 'L' or '<' means turn left
•             Serial.println("Left");
•             turnLeft();
•             break;
•
•         case 'R': case '>': // 'R' or '>' means turn right
•             Serial.println("Right");
•             turnRight();
•             break;
•
•         case 'S': case '*': // 'S' or '*' means stop
•             Serial.println("Stop");
•             stopMotion();
•             break;
•
•         default: // Any other char → emergency stop
•             stopMotion();
•             break;
•     }
• }
• }
```

```
• // Continuously check distance ahead and steer around obstacles
• void autonomousObstacleAvoidance() {
•     int frontDistance = readUltrasonic();    // Measure distance ahead
•     if (frontDistance <= 12) {                // If obstacle is closer than 12 cm
•         stopMotion();                        // Stop forward motion
•         moveBackward();                      // Back up slightly
•         delay(100);
•         stopMotion();
•
•         // Scan left and right distances by moving servo
•         int leftDistance  = scanAt(LEFT_ANGLE);
•         int rightDistance = scanAt(RIGHT_ANGLE);
•
•         smoothServoWrite(CENTER_ANGLE);      // Re-center servo
•         delay(200);
•
•         // Turn toward the side with more free space
•         if (leftDistance > rightDistance) {
•             Serial.println("Turning left");
•             turnLeft();
•         } else {
•             Serial.println("Turning right");
•             turnRight();
•         }
•         delay(TURN_DURATION_MS);             // Run turn long enough for ~90°
•         stopMotion();
•         delay(200);
•     }
•     else {
•         moveForward();                       // No obstacle → keep moving forward
•     }
• }
•
• // Trigger ultrasonic pulse and read echo time, return distance in cm
• int readUltrasonic() {
•     digitalWrite(TRIG_PIN, LOW);
•     delayMicroseconds(4);
•     digitalWrite(TRIG_PIN, HIGH);
•     delayMicroseconds(10);
•     digitalWrite(TRIG_PIN, LOW);
•     long duration = pulseIn(ECHO_PIN, HIGH); // Measure echo pulse width
•     return duration / 29 / 2;                // Convert microseconds to cm
• }
```

```
•
• // Move servo to a specified angle, wait, then measure distance
• int scanAt(int angle) {
•     smoothServoWrite(angle);           // Rotate servo smoothly
•     delay(SCAN_DELAY_MS);              // Allow servo to settle
•     return readUltrasonic();            // Return measured distance
• }
•
• // Smoothly increment or decrement servo until reaching targetAngle
• void smoothServoWrite(int targetAngle) {
•     int current = servo.read();         // Get current servo angle
•     while (current != targetAngle) {
•         if (current < targetAngle) current++;
•         else current--;
•         servo.write(current);           // Move one degree at a time
•         delay(5);                       // Small delay for smooth motion
•     }
• }
•
• // Drive all four motors forward
• void moveForward() {
•     motor1.run(FORWARD); motor2.run(FORWARD);
•     motor3.run(FORWARD); motor4.run(FORWARD);
• }
•
• // Drive all four motors backward
• void moveBackward() {
•     motor1.run(BACKWARD); motor2.run(BACKWARD);
•     motor3.run(BACKWARD); motor4.run(BACKWARD);
• }
•
• // Spin in place to the right: left wheels backward, right wheels forward
• void turnRight() {
•     motor1.run(BACKWARD); motor2.run(BACKWARD);
•     motor3.run(FORWARD); motor4.run(FORWARD);
• }
•
• // Spin in place to the left: left wheels forward, right wheels backward
• void turnLeft() {
•     motor1.run(FORWARD); motor2.run(FORWARD);
•     motor3.run(BACKWARD); motor4.run(BACKWARD);
• }
•
```

```
• // Stop all motors immediately
• void stopMotion() {
•     motor1.run(RELEASE); motor2.run(RELEASE);
•     motor3.run(RELEASE); motor4.run(RELEASE);
• }
•
```

- **Testing results and debugging via terminal**

To validate the functionality of our voice-controlled, RC command-controlled and obstacle-avoiding robotic car, we have prepared demonstration videos showcasing the three main features of the system. Each video highlights aspects of the car's operation and how it responds to user input and environmental conditions. Below are the links and explanations corresponding to each demo video.

1. Voice Control Demonstration

In these videos, we demonstrate the voice command capability of the robotic car. Using a smartphone voice recognition application which is Arduino BlueControl, we gave verbal commands such as:

"Go" → The car moves forward

"Back" → The car moves backward

"Left" → The car turns left

"Right" → The car turns right

"Stop" → The car halts immediately

The car receives these commands via Bluetooth and responds in real-time, showcasing the integration of speech recognition with the embedded motor control system. This feature offers a hands-free control mechanism, making the system especially useful for individuals with limited mobility.

Links to videos with the result :

https://istinye-my.sharepoint.com/:v:/g/personal/210901038_stu_istinye_edu_tr/EdbA3dY4Sh5LnX5B5h9uuxgBD3hFwqPUG_xiSeq6Wd6sQg?e=fDUove

https://istinye-my.sharepoint.com/:v:/g/personal/210901038_stu_istinye_edu_tr/Ec5eDj6AKCtDruNDsj4tLP8BL12cUQJRjZaWA9I2cEoTPQ?e=xyvmh9

https://istinye-my.sharepoint.com/:v:/g/personal/210901038_stu_istinye_edu_tr/EV1hSvOOM4ZHjNe_QoXd99QBga31-LXje2_ik_iyccbztw?e=ytmDZo

2. Obstacle Avoidance Demonstration

This video highlights the autonomous obstacle avoidance system of the robotic car. The HC-05 ultrasonic sensor mounted on a servo motor continuously scans the front, left, and right directions. When the sensor detects an object within 12 cm, the car automatically:

Stops and reverses briefly to avoid collision

Rotates the servo to scan left and right directions

Decides the direction with more space

Turns toward the obstacle-free side and resumes forward motion

This behavior demonstrates the car's ability to navigate complex environments without human intervention, making it suitable for autonomous applications.

Links to videos with the result:

https://istinye-my.sharepoint.com/:v:/g/personal/210901038_stu_istinye_edu_tr/EUxrRUsi5QZAKBE1rk2C5TIBgr5VT4504ZfJFydlzrPocQ?e=uHeAjv

https://istinye-my.sharepoint.com/:v:/g/personal/210901038_stu_istinye_edu_tr/ES7LfOXSQ4NMjFBfc6wwPhkB-E4dV-xk4Hx36KpczXnVEw?e=Sp2kay

3. RC Command Control Demonstration

In this demo, we control the car using an RC-style interface called RC Bluetooth Controller. Each button corresponds to a specific command:

Forward

Backward

Turn Left

Turn Right

When buttons are pressed, the car executes the corresponding movement immediately. This showcases the system's manual control mode, allowing for precise driving and testing. It also validates the reliability of the Bluetooth communication and motor control responses.

Link to video with the result:

https://istinye-my.sharepoint.com/:v/g/personal/210901038_stu_istinye_edu_tr/EcfI7Dr5fBtNijSTj8aWazkB-A1kogPZzn33xD93hjqZYw?e=sxCROW

Each of these demo videos clearly illustrates the core functionalities of our project and collectively proves that the system operates effectively under all three control scenarios: voice-driven, autonomous, and manual.

- **Any modifications made during the optimization process**

- 1. Servo scan angles**

- **Original:** rightsee at 20° , leftsee at 180° .
 - **Trials:**
 - `#define SERVO_LEFT 160 / #define SERVO_RIGHT 30`
 - $135^\circ / 45^\circ$
 - **Final:** settled on 150° (left) and 30° (right) for the best balance between field of view and mechanical limits.

- 2. Obstacle-detection threshold**

- **Tested distances:** 12 cm, 18 cm, 20 cm
 - **Chosen value:** 12 cm gave the most reliable avoidance without excessive early turning.

- 3. Speed and timing constants**

- 3.1 Servo scan delay (SCAN_DELAY_MS = 400 ms)**

- Typical analog servos spin at $\approx 300^\circ/\text{s} \rightarrow \approx 3.3 \text{ ms per degree}$.
 - We sweep $\pm 60^\circ$ ($30^\circ \rightarrow 150^\circ$), so $60^\circ \approx 200 \text{ ms}$.
 - Doubling to 400 ms gives ample settling time and lets the ultrasonic echo stabilize, cutting down on noisy readings.

- 3.2 Turn duration (TURN_DURATION_MS $\approx$ 450 ms)**

- Forum baseline: 90° spin at full PWM (255) ≈ 300 ms.
- Our PWM=180 is $\approx 70\%$ of 255, so ideal time $\approx 300 \text{ ms} \times (180/255) \approx 212$ ms.
- Real-world factors (battery sag, friction, inertia) demand extra margin: doubled to ≈ 425 ms and rounded up to **450 ms**.

3.3 Motor speed (MOTOR_SPEED = 180 PWM)

- **Tested at 255** (100% duty) on a 4×AA pack (~ 6 V):
 - Led to severe voltage sag/brown-outs under load.
 - Caused servo glitches and repeated resets during high-torque maneuvers.
 - Dramatically cut battery runtime to only a few minutes.
- **Settled on 180 PWM** ($\sim 70\%$ duty) to balance torque, top speed, and stable voltage under load.

4. Calibration & safety margin

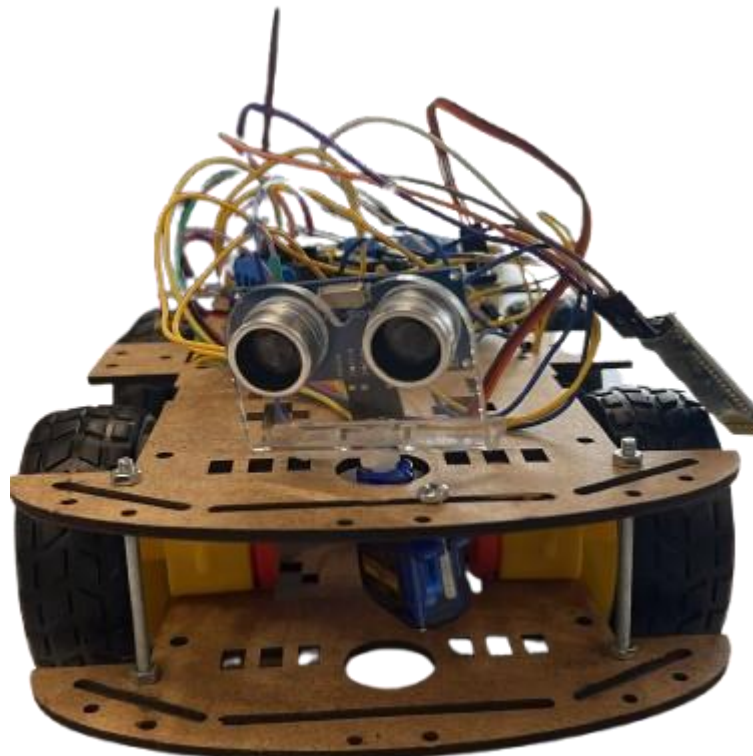
These values were established and fine-tuned during on-vehicle testing by $\pm 10\text{--}20\%$:

- **SCAN_DELAY_MS** was adjusted to eliminate any servo bounce or echo jitter.
- **TURN_DURATION_MS** was tweaked until 90° turns were consistently accurate.
- **MOTOR_SPEED** was modified up or down to balance responsiveness with battery life.

Appendix A – Complete Requirements Matrix

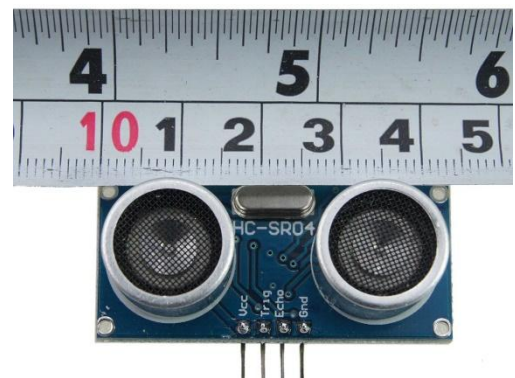
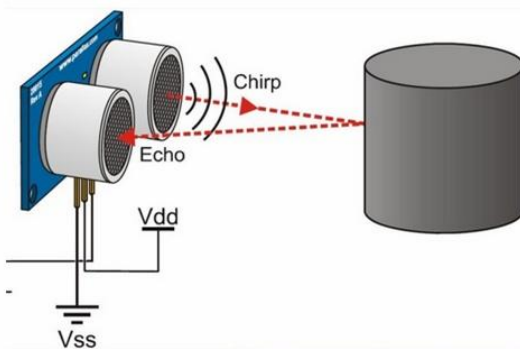
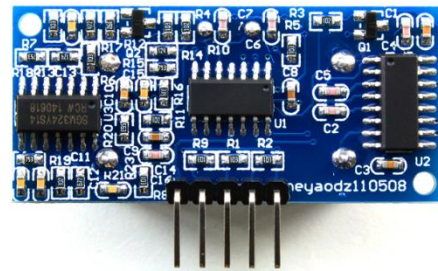
<i>ID</i>	Type	Requirement Description	Verification Method
<i>FR1</i>	Functional Requirement	The robot shall move forward, backward, left, and right using voice commands.	Test voice commands and observe
<i>FR2</i>	Functional Requirement	The robot shall receive and execute Bluetooth commands from a mobile app.	Pair mobile app and test commands
<i>FR3</i>	Functional Requirement	The robot shall stop or change direction upon detecting an obstacle using an ultrasonic sensor.	Place obstacles; observe behavior
<i>FR4</i>	Functional Requirement	The robot shall operate in autonomous navigation mode, avoiding obstacles.	Activate mode; test navigation
<i>FR5</i>	Functional Requirement	The robot shall switch between manual and autonomous modes upon receiving a specific “mode” command.	Send mode-switch command; verify behavior
<i>FR6</i>	Functional Requirement	The robot shall rotate its ultrasonic sensor via servo from LEFT_ANGLE to RIGHT_ANGLE before avoidance.	Monitor servo sweep angles on each obstacle encounter
<i>FR7</i>	Functional Requirement	The robot shall immediately stop upon receiving an emergency stop signal.	Trigger emergency stop; ensure motors release instantly
<i>NFR1</i>	Non-Functional Requirement	The system shall operate within a voltage range of 5–7 V DC.	Measure input voltage range
<i>NFR2</i>	Non-Functional Requirement	Obstacle detection response time shall be less than 600 ms (including servo sweep).	Measure end-to-end response with an oscilloscope
<i>NFR3</i>	Non-Functional Requirement	The robot shall operate continuously for at least 15 minutes on a fully charged battery.	Run endurance test
<i>NFR4</i>	Non-Functional Requirement	The voice recognition module shall respond to commands with at least 85% accuracy in a quiet environment.	Test recognition accuracy

<i>NFR5</i>	Non-Functional Requirement	The robot's total weight shall be less than 1.5 kg to ensure mobility and stability.	Weigh assembled robot
<i>NFR6</i>	Non-Functional Requirement	The robot shall operate reliably between 0 °C and 40 °C ambient temperature.	Environmental chamber or field test
<i>HSI1</i>	Hardware-Software Interface	The Arduino shall interface with the L293D motor driver to control the DC motors.	Check wiring and control signals
<i>HSI2</i>	Hardware-Software Interface	The Arduino shall read obstacle distance from the ultrasonic sensor (e.g., HC-SR04).	Read sensor data via serial output
<i>HSI3</i>	Hardware-Software Interface	The Arduino shall receive Bluetooth commands via the HC-05 module.	Test Bluetooth communication
<i>HSI4</i>	Hardware-Software Interface	The Arduino shall send PWM control signals to the servo on pin SERVO_PIN for obstacle scanning.	Verify PWM output on oscilloscope or logic analyzer
<i>HMI1</i>	Human-Machine Interaction	Users shall give commands through voice or Bluetooth using a mobile app.	Demonstrate both control methods



Sir Avoids-a-Lot

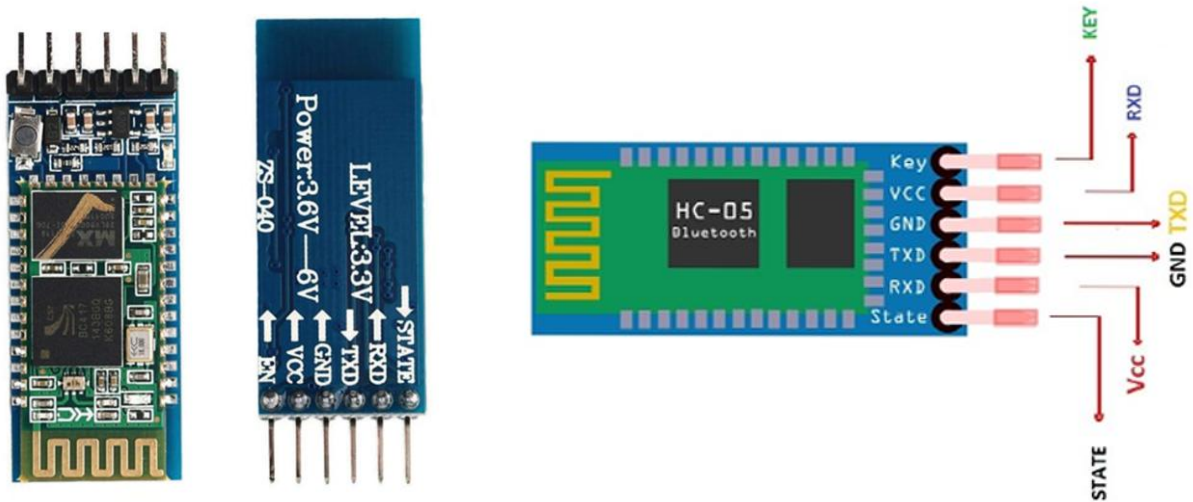
Appendix B - Datasheets of Sensors (HC-05 and Ultrasonic Sensor) and Motor Shield



Ultrasonic Sensor (HC-SR04) Datasheet

- **Operating voltage:** 5V DC
- **Measurement range:** 2cm – 400cm
- **Accuracy:** $\pm 3\text{mm}$
- **Trig pulse width:** $10\mu\text{s}$
- **Echo pulse width:** proportional to distance

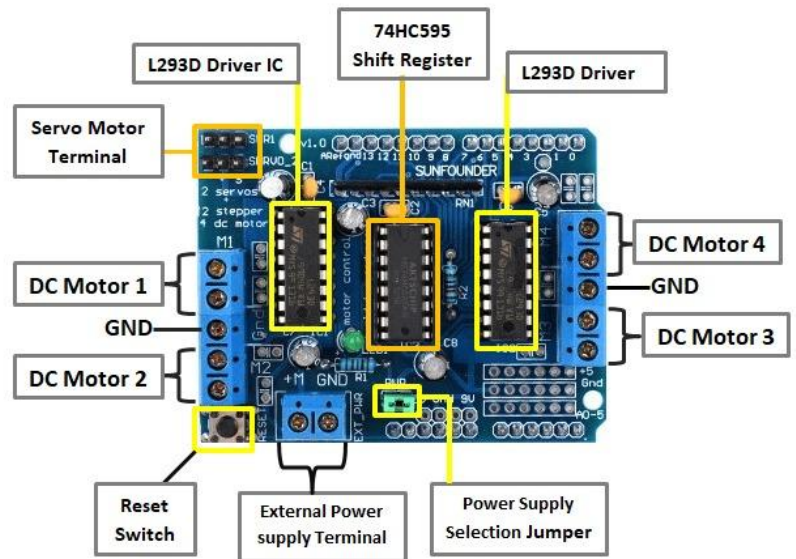
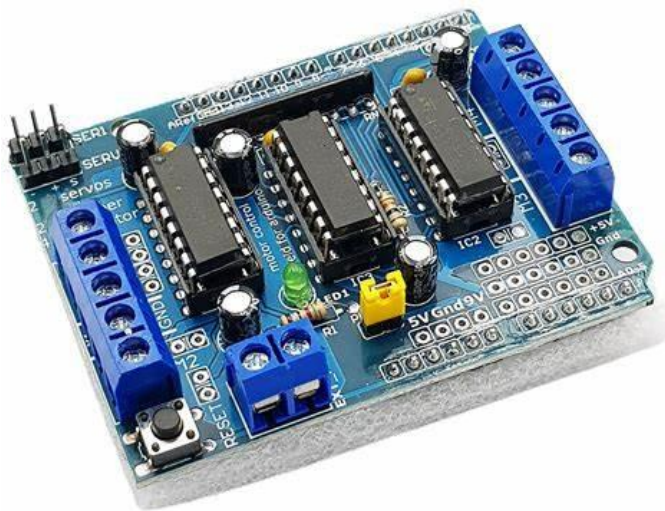
[HC-SR04 Ultrasonic Sensor Working, Pinout, Features & Datasheet](#)



HC-05 Bluetooth Module Datasheet

- **Voltage:** 3.3V to 5V
- **Baud rate:** 9600 (default) for data communication and 38400 for command mode communication.
- **Communication:** Serial (TX/RX)

[HC-05 Bluetooth Module Pinout, Datasheet, Features & Applications - The Engineering Projects](#)



L293D Motor Shield – Technical Specifications

- **Motor Driver IC:** L293D (Quadruple Half-H Driver)
- **Operating Motor Voltage:** 4.5V to 36V
- **Logic Voltage:** 5V (supplied via Arduino)
- **Maximum Current:** 600 mA per channel (peak up to 1.2 A)
- **Motor Control Capability:**
 - Supports up to 4 DC motors (bidirectional control)
 - Supports up to 2 stepper motors
- **Speed Control:** Pulse Width Modulation (PWM) supported
- **External Power Supply:** Screw terminal for motor power (recommended 6–12V)
- **Control Interface:** Digital I/O pins (IN1–IN4, ENA, ENB)
- **Servo Support:** Two servo headers (Arduino pins 9 and 10)
- **Compatibility:** Designed for Arduino Uno, Mega, and similar boards

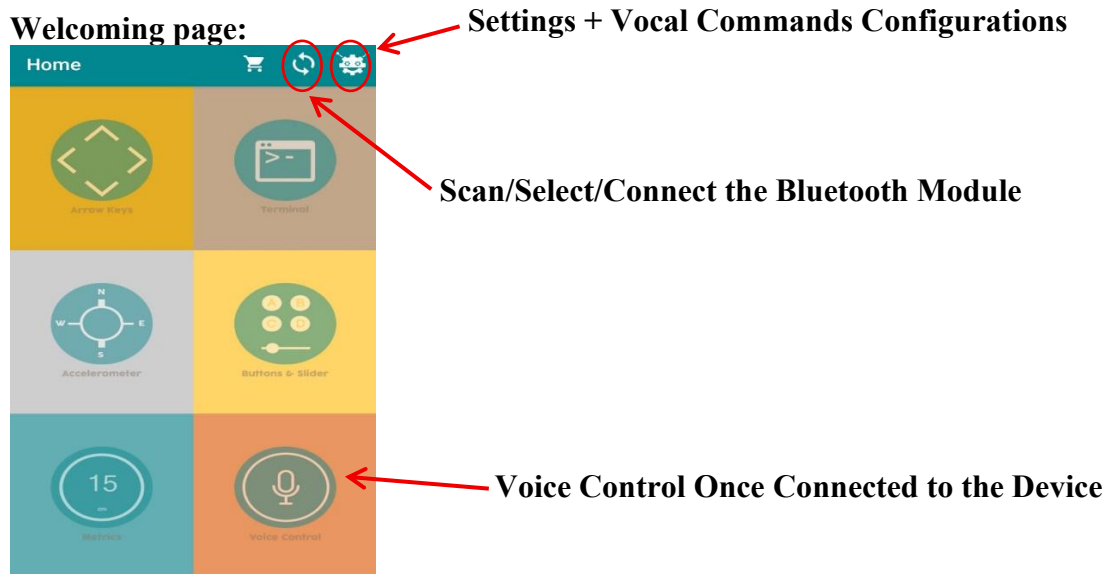
[L293D Motor Shield.pdf](#)

Appendix C – Applications Used for Arduino Bluetooth Control

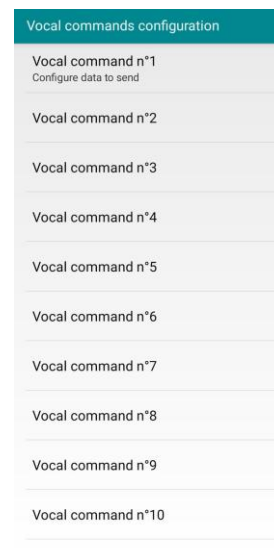
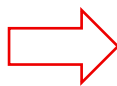
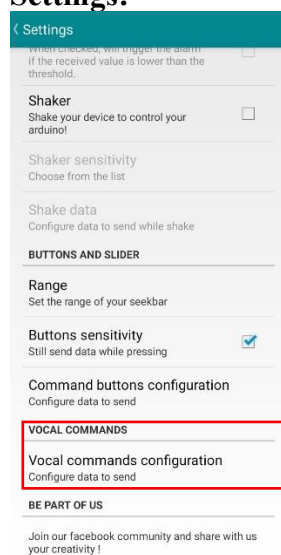
1. Arduino BlueControl

(<https://play.google.com/store/apps/details?id=com.broxcodex.arduinoobluetoothfree>)

With Arduino Bluetooth Control, we can customize our own voice commands and use them to control all microcontroller-based boards. We obtained the commands specified in our code using these configurations.



Settings:





Each Command and Its Configuration: Voice Command & Data to Send

Go:

Vocal command n°1

Vocal command

Type the vocal command to pronounce

Data to send

Set the corresponding data to send

Vocal command

go

CancelOK

Vocal command n°1

Vocal command

Type the vocal command to pronounce

Data to send

Set the corresponding data to send

Data to send to arduino

A

CancelOK

Back:

Vocal command n°2

Vocal command

Type the vocal command to pronounce

Data to send

Set the corresponding data to send

Vocal command

back

CancelOK

Vocal command n°2

Vocal command

Type the vocal command to pronounce

Data to send

Set the corresponding data to send

Data to send to arduino

-

CancelOK

Left:

Vocal command n°3

Vocal command

Type the vocal command to pronounce

Data to send

Set the corresponding data to send

Vocal command

left

CancelOK

Vocal command n°3

Vocal command

Type the vocal command to pronounce

Data to send

Set the corresponding data to send

Data to send to arduino

<

CancelOK

Right:

Vocal command n°4

Vocal command

Type the vocal command to pronounce

Data to send

Set the corresponding data to send

Vocal command

right

CancelOK

Vocal command n°4

Vocal command

Type the vocal command to pronounce

Data to send

Set the corresponding data to send

Data to send to arduino

>

CancelOK

Stop:

Vocal command n°5

Vocal command
Type the vocal command to pronounce

Data to send
Set the corresponding data to send

Vocal command

stop

Cancel OK

Vocal command n°5

Vocal command
Type the vocal command to pronounce

Data to send
Set the corresponding data to send

Data to send to arduino

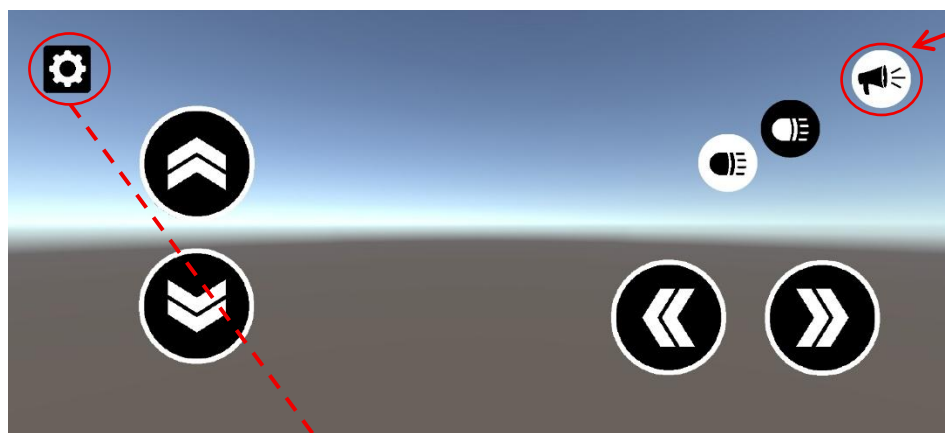
*

Cancel OK

2. RC Bluetooth Controller

(<https://play.google.com/store/apps/details?id=com.SanitTech.RCBluetoothControllerHC05>)

RC Bluetooth Controller (HC-05) is an app designed to control an RC car using the HC-05 Bluetooth module. This app provides Arduino source code and allows you to control your RC car via an Android device. It has the simplest user interface and did not cause any problems during use.



Scan/Select the Bluetooth Module

